

SynthEval: Learned-NER De-identification, Construction-True Labeling, and Operational Hardening

A Technical Addendum to *SynthEval Studio and Cloud: Quality-Gated Synthetic Data Generation*

Patrick Hill, M.S.
Founder & Principal AI/ML Systems Architect
PBH Applied Systems LLC

June 2026

Abstract

This addendum extends the June 2026 SynthEval methodology paper rather than replacing it. The original paper (Hill, 2026) documented a dual-substrate, quality-gated pipeline for synthetic instruction data; the results reported there stand unchanged. This document records four developments made since: a construction-true de-identification product whose span labels are correct by construction rather than inferred by a detector; a learned-NER premium evaluation tier that closes the regex detector’s documented recall gap (Hill, 2026) on free-text entities; a template-leak rejection gate that removes a class of emergent generation defect at the source; and an operational-hardening effort that diagnosed and permanently resolved a hardware fault affecting the local substrate. Each contribution is presented in the measured register of the original: results are reported as measured, bounds are stated plainly, and the single design decision that most shaped the de-identification tier — restricting the learned model to the entity types the regex detector cannot serve — is told as the empirical narrative it was, including the configuration that failed. The de-identification premium tier reaches an F1 of 0.958–0.965 across both substrates while recovering free-text-entity recall from zero, and the hardening effort restored steady-state cloud generation cost to its measured figure of approximately \$0.47 per thousand records. A closing section documents the delivery and provenance machinery — the versioned run manifest (schema 1.2), the reconciliation cross-check, and the checksum-sealed delivery package — that makes each of these claims auditable rather than merely asserted.

Contents

1 Relationship to the June 2026 Methodology Paper	2
2 The Construction-True De-identification Product	3
2.1 Two Labeling Modes	3
2.2 Why Construction-True Labeling Is the Differentiator	4
2.3 Record Shape	4
3 The Learned-NER Premium Evaluation Tier	4
3.1 The Detector Factory	4
3.2 The Gap-Only Design Decision	5
3.3 Validated Results	6
3.4 Per-Type Detail	6

4	The Template-Leak Rejection Gate	7
4.1	The Defect	7
4.2	Why Prevention by Prompt Fails	7
4.3	Rejection as Prevention	7
4.4	Validation	8
5	Operational Hardening: Diagnosing and Resolving a Substrate Fault	8
5.1	The Symptom	9
5.2	Isolating Hardware from Software	9
5.3	Root Cause and Permanent Resolution	9
5.4	The Fault Was Also a Cost	10
6	Cloud Economics After Consolidation	10
6.1	Consolidation Did Not Raise Per-Record Cost	10
6.2	Measured Steady-State Cost	10
6.3	The Resume Tax, and Why the Last Run Is the Honest Figure	11
7	Delivery, Manifest, and Provenance	11
7.1	The Run Manifest as the Provenance Record	11
7.2	The Delivery Packager	12
7.3	Why This Is the Product	12
8	Limitations	12
8.1	The Premium Tier’s Precision Cost Is Real and Concentrated	12
8.2	The Gap Set Is Empirical, Not Derived	13
8.3	The Template-Leak Pattern Is Calibrated, Not Complete	13
8.4	The Hardware Resolution Is Verified for This Host	13
9	Conclusion	13
10	References	14

1 Relationship to the June 2026 Methodology Paper

This is an addendum, and the distinction matters. The original paper — *SynthEval Studio and Cloud: Quality-Gated Synthetic Data Generation* (Hill, 2026) — remains the canonical description of the system: the two-axis architecture, the dual-signal hallucination gate, the compliance and masking layers, the substrate economics, and the limitations that bound every claim. Nothing in this document revises those results. The measured zero residual PII leak rate, the cross- substrate quality parity, and the gate behavior reported there (Hill, 2026) are unchanged, and a reader encountering SynthEval for the first time should begin with the original paper, not this one.

What follows documents work done after that paper was published. The choice to write a second, focused document rather than revise the first is deliberate, and it reflects the same principle the original paper argues for (Hill, 2026): provenance is preserved by versioned, append-only documentation, not by silently editing a released artifact. The original paper is linked from a published dataset and has been read in its released form; rewriting it underneath those references would be precisely the kind of opacity the methodology is designed to avoid. A reader who wants to know what the system did in June 2026 reads the original; a reader who wants to know what was added afterward reads this. The two together form the record.

The developments documented here are additive. They extend the de-identification product line from a regex-labeled primitive into a construction-true, learned-NER-evaluated product;

they add one generation-time quality gate; and they record an operational fault and its permanent resolution. None of them invalidates a prior result. Where a number from this work can be compared to one in the original paper (Hill, 2026) — most directly in the cloud economics — the comparison confirms the original measurement rather than correcting it, as Section 6 describes.

The internal mechanics referenced throughout are documented in the repository’s `architecture.md`, `repo_map.md`, and the run-manifest schema (now at version 1.2), which were updated alongside this work and are the authoritative source for file-level detail.

2 The Construction-True De-identification Product

The primary product of the original paper (Hill, 2026) is instruction data: `(instruction, input, output)` tuples evaluated for diversity, quality, and compliance. The de-identification product is a different primitive — the span-labeled pair `(text, spans)`, where each span records the character range, type, and value of a personal-data entity in the text. This is the unit a buyer needs to train or benchmark a de-identification model: text with known, labeled PII locations.

What distinguishes the product is not the primitive but the provenance of the labels. There are two ways to obtain the spans, and the system supports both under the `labeling` configuration axis.

The de-identification product is the second of SynthEval’s two orthogonal axes. The *substrate* axis — local Studio versus Modal Cloud — varies by package and governs *where* generation runs; the *product* axis varies by configuration (`output.product / output.labeling`) and governs *what* is produced. Both products share the entire `syntheval_core` pipeline — gating, checkpointing, provenance, evaluation, and delivery — and differ only at the emitter seam. A single factory, `make_emitter(config)`, selects the record shape without forking the orchestrator: `InstructionEmitter` for the instruction product, `DeIdConstructionTrueEmitter` and `DeIdRegexEmitter` for the two de-identification labeling modes. The de-identification line itself lives under `syntheval_core/deid/` — `injector.py` (construction-true placement), `providers.py` (per-type value generators), `emitters.py` (the record emitters), and `evaluation.py` (the span-scoring suite).

2.1 Two Labeling Modes

Regex-derived labeling (`labeling: regex_derived`) runs the production regex masker’s `detect_spans` over generated text and records what it finds. This is honest but bounded: the labels inherit the detector’s recall ceiling. Any entity the regex cannot match — a free-text personal name, an institution identifier with no fixed shape — is absent from both the text’s masking and its labels. A model trained on regex-derived labels learns to find what regex already finds, and no more. The regex-derived set is therefore most useful as a faithful, free, representative sample: it shows the format and the structured-entity coverage without claiming to label what regex cannot see.

Construction-true labeling (`labeling: construction_true`) inverts the direction of the ground truth. Rather than generating text and then detecting entities in it, the injector (`injector.py`) takes gated, placeholder-bearing output and substitutes format-shaped synthetic values into the placeholder tokens, drawing from the per-type generators in `providers.py`. Because the injector *places* each value, it knows the exact character range, type, and value of every entity it wrote — and it logs those as the spans. The labels are correct by construction: they come from the injection log, not from a detector. There is no recall ceiling to inherit, because nothing is being detected.

2.2 Why Construction-True Labeling Is the Differentiator

This is the product’s defensible core. A regex-labeled de-identification dataset can only ever be as complete as the regex that labeled it; its ground truth and its detector share a blind spot, so it cannot be used to measure whether a model finds entities the regex misses — the very capability a buyer building a de-identification model most needs to evaluate. Construction-true labeling breaks that coupling. The ground truth is established at injection time by generators that produce free-text names, institution identifiers, and structured values alike, so the labels cover entity types no regex detector reaches. A model evaluated against construction-true spans is measured against what was actually placed in the text, not against what a particular detector happened to find.

The synthetic origin of the values is also what makes the product safe to sell into regulated contexts without the re-identification analysis a real-data de-identification corpus would require. The values are generated, not collected; no real personal data is present at any point, because none ever entered the pipeline. The construction-true span log documents exactly what synthetic PII was placed and where, which is both the training label and the privacy attestation.

2.3 Record Shape

Both labeling modes emit the same record shape through their respective emitters, so a consumer sees a uniform structure regardless of provenance:

```
{ "text": ..., "spans": [ { "start", "end", "type", "value", "source" } ],  
  "masked_text": ..., "category": ... }
```

The `source` field on each span is the provenance marker: `injected` for a construction-true span (ground truth from the injection log), `regex` or `gliner` for a detector-derived span. The evaluation in Section 3 scores detector spans against the injected ground truth, and the `source` field is what makes that scoring auditable per span.

3 The Learned-NER Premium Evaluation Tier

The regex detector described in the original paper (Hill, 2026) has a known, documented property: near-perfect precision on structured PII — email addresses, IP addresses, dates, account and order identifiers, medical codes — and a recall gap on free-text entities it cannot pattern-match, principally personal names and institution identifiers. The original paper treats this gap as a deliberate, disclosed artifact (Hill, 2026): a buyer’s own learned model is expected to close it. The premium evaluation tier documented here is the instrument that measures how well a learned model does so, and it is itself built from a learned model.

3.1 The Detector Factory

Evaluation detector selection is a single configuration key, `evaluation.detector`, resolved through a factory in the evaluation registry (`build_deid_detector / build_deid_pipeline` in `evaluation/registry.py`) and overridable per run with a `--detector` command-line flag. Because the orchestrator’s auto-evaluation, the CLI `evaluate` path, and the delivery packager all construct the detector through this one factory, the `evaluation.detector` key drives every de-identification scoring uniformly. The committed configuration ships the base product (`regex`); the premium tier is selected per run, so the learned-model weights are never required on the base path.

evaluation.detector	Detector	Role
regex (default)	PIIMasker.detect_spans	Base product: hand-written regex, near-perfect precision on structured PII, documented recall gap on free-text entities.
hybrid	regex + GLiNER2, gap-only	Premium tier: the learned model runs only on the gap entity types, closing the free-text recall gap without disturbing structured-PII precision.
gliner	GLiNER2 alone, full label set	Standalone scoring of the learned model’s coverage across all types.

The learned model is GLiNER2 (Zaratiana et al., 2024, 2025) (fastino/gliner2-large-v1, a DeBERTaV3 (He et al., 2020, 2021) encoder), wrapped by GlinerDetector and composed with the regex masker by HybridDetector (both in deid/gliner_detector.py), loaded lazily so the base path imports nothing heavy. In hybrid mode the regex detector claims spans first and wins any overlap, preserving its precision; the learned model fills only the regions and types regex leaves unserved.

3.2 The Gap-Only Design Decision

The most consequential decision in this tier was how much of the labeling task to give the learned model. The decision was made empirically, and the path included a configuration that failed instructively. It is reported here in full because the failure is as informative as the result.

Full ensemble (the configuration that failed). The first design let GLiNER2 predict every entity type and merged its spans with the regex detector’s. On a 107-record run this collapsed micro-precision from the regex base of roughly 0.99 to **0.70**. The cause was specific and measurable: the learned model re-predicted the structured types the regex already handled perfectly — dates, emails, IPs, order identifiers — at slightly different offsets or on values the regex had correctly declined, and those redundant predictions landed as false positives. Of 142 false positives in that run, 127 were on structured types the regex already detected at perfect precision. The learned model was not finding new errors; it was manufacturing them on solved problems.

Gap-only (the correction). The fix was to restrict the learned model to the entity types the regex detector cannot serve — the free-text gap types — and let regex own everything else. With the learned model requesting only the gap labels, it can no longer produce a structured-type false positive, because it no longer predicts structured types at all. Precision returned to the regex base while the free-text recall gap closed.

The second narrowing. The initial gap set included the two numeric types whose narrow regex patterns under-detect: account numbers and SWIFT/BIC codes. A thousand-record run showed this was a mistake of a different kind. The learned model scored *worse* than regex on account numbers — recall 0.30 against the regex base’s 0.47 — while adding false positives, and tagged unrelated tokens as SWIFT/BIC codes at 0.20 precision. The learned model was not better than regex on these numerics; it was simply noisier. They were removed from the gap set, returning to the regex-documented gap where they belong. The final gap set is the

two entity types the learned model genuinely recovers and regex genuinely cannot: personal names (STUDENT_NAME) and institution identifiers (SCHOOL_ID).

The general principle the two narrowings establish is that a learned model earns its place in the ensemble only on the types where it strictly outperforms the deterministic detector, and that the correct scope is found by measurement, not by assuming more model coverage is better. On this product, more coverage was worse twice before it was right.

3.3 Validated Results

With the gap-only hybrid, the premium tier was validated on full thousand-record runs across both substrates. The results are consistent across substrate, which matters: the quality parity the original paper establishes for generation (Hill, 2026) holds for the learned-NER evaluation as well.

Configuration	Precision	Recall	F1
Regex base (Cloud, 1,095 records)	0.986	0.921	0.952
Full-ensemble hybrid (the failed configuration)	0.700	0.979	0.816
Gap-only hybrid (Studio, 1,007 records)	0.955	0.975	0.965
Gap-only hybrid (Cloud, 1,092 records)	0.947	0.969	0.958

The premium tier exceeds the regex base on F1 (0.958-0.965 against 0.952) — it is not a precision-for-recall trade but a strict improvement — while recovering the gap types from zero recall: STUDENT_NAME from 0.00 to 0.90-0.98 and SCHOOL_ID from 0.00 to 0.95-1.00 across the validated runs.

3.4 Per-Type Detail

The full per-type breakdown for the representative Cloud hybrid run (run_cloud_20260621_163329, 1,092 records, 3,599 ground-truth spans) shows the design working exactly as intended: every structured type the gap-only restriction protects carries zero learned-model false positives, while the two gap types carry the recovery and its honest cost.

Type	Supp.	P	R	F1	TP	FN	FP
ORDER_ID	623	1.00	0.99	1.00	619	4	0
EMAIL	542	1.00	1.00	1.00	542	0	0
DATE	436	1.00	0.98	0.99	428	8	0
IP_ADDRESS	354	1.00	1.00	1.00	354	0	0
NATIONAL_ID	343	0.86	1.00	0.93	343	0	55
URL	234	1.00	1.00	1.00	234	0	0
CPT_CODE	203	1.00	0.99	0.99	200	3	0
STUDENT_NAME	150	0.54	0.90	0.68	135	15	114
LICENSE_PLATE	145	1.00	0.99	1.00	144	1	0
ICD10_CODE	135	1.00	0.96	0.98	129	6	0
SCHOOL_ID	112	0.80	0.96	0.87	107	5	27
ACCOUNT_NO	96	1.00	0.42	0.59	40	56	0
MAC_ADDRESS	84	1.00	1.00	1.00	84	0	0
CREDIT_CARD	70	0.99	1.00	0.99	70	0	1
IBAN	38	1.00	1.00	1.00	38	0	0
ZIP	30	1.00	0.70	0.82	21	9	0
SWIFT_BIC	4	0.00	0.00	0.00	0	4	0

Three features of this table carry the design. First, every structured type the gap-only restriction protects — order identifiers, email, dates, IPs, URLs, CPT and ICD-10 codes, license plates, MAC addresses, IBANs — shows zero false positives; the learned model never touches them. Second, the gap types carry the recovery: `STUDENT_NAME` and `SCHOOL_ID` are found at 0.90 and 0.96 recall, where the regex base finds none. Third, the precision cost is concentrated, visible, and honest: the 114 and 27 false positives on the two gap types are the learned model’s over-tagging of names and institution identifiers — 141 false positives to recover entities the regex cannot find at all. The remaining structured false positives, the 55 on `NATIONAL_ID`, are a property of the regex detector’s numeric breadth disclosed in the original paper (Hill, 2026), not of the learned model. `ACCOUNT_NO` and `SWIFT_BIC`, returned to regex-only after the second narrowing, sit cleanly in the documented gap with no learned-model noise.

4 The Template-Leak Rejection Gate

The gating pipeline of the original paper (Hill, 2026) rejects records on hallucination, privacy, bias, GDPR-surface, URL, and fidelity grounds. This work adds one more rejection reason, and the reason it was needed illustrates a general property of generation defects that prompt engineering cannot reach.

4.1 The Defect

In instruction generation, the model occasionally emits a fill-in-the-blank stub in place of content: a token such as `<INSERT_PATIENT_IDS_HERE>` or `<INSERT_URL>` standing where a real value or passage should be. These are not placeholders the templates contain; the instruction the model receives never mentions them. They are emergent — the model, asked to produce a document, sometimes produces a template skeleton of one instead, inventing a stand-in token where it has “decided” to leave a blank. A record containing such a stub is a failed generation: the surrounding text is a template, not the document it was asked to write.

The defect is rare. In a representative production run it appeared in two records out of roughly a thousand. But it is a visible quality flaw in a sold artifact, and its rarity is precisely what makes it hard to address by the obvious means.

4.2 Why Prevention by Prompt Fails

The intuitive fix is to instruct the model not to emit such tokens. This does not work, for three reasons that are worth stating because they generalize. First, the tokens are not in any template or instruction to remove; they originate in the model’s generation, so there is no source-side text to correct. Second, negative instructions of the form “never emit a token like X” are unreliable against rare emergent behavior and can prime the very pattern they forbid. Third, the space of possible stub forms cannot be enumerated in advance — the model might invent `<INSERT_URL>`, `<SOMETHING_HERE>`, or a shape not yet seen — so a prompt listing forbidden forms is always incomplete. Prevention by prompt is the wrong instrument for a rare, emergent, open-ended artifact.

4.3 Rejection as Prevention

The reliable fix is detection at the point the record is finalized, treating a template-leak stub as a quality failure on the same footing as a hallucination or a compliance violation. The gate matches the stub class — the `INSERT_*` and `*_HERE` token forms — on the raw model output, rejects any record that contains one, tallies the rejection under a new `template_leak` reason, and lets overgeneration backfill the rejected record with a clean one. This is prevention in the

sense that matters: the defect never reaches the delivered dataset, even though the model still occasionally produces it.

The decision to reject rather than repair is deliberate. A record containing `<INSERT_PATIENT_IDS_HERE>` is not a good document with one bad token; it is a template skeleton whose surrounding sentence is itself a stub. Stripping or substituting the token would leave a still-broken sentence. Rejecting the record removes the failed generation entirely, and overgeneration — already part of the pipeline — replaces it with a clean one at no change to the delivered count.

The gate is deliberately narrow. It matches only the fill-in-the-blank stub class and leaves untouched the legitimate placeholder forms a good instruction output may contain: enumerated entities that disambiguate multiple instances of one type (`DATE_1`, `CPT_CODE_2`), domain-realistic identifiers (`NPI_1`), and descriptive field placeholders a document genuinely calls for (`CONSENT_STATUS`, `DATE_ACCESS_REQUEST`). Distinguishing the defect from these legitimate forms was the point: a broader filter that collapsed all bracketed tokens would degrade good outputs to remove a two-in-a-thousand flaw.

4.4 Validation

The gate was validated against the run that surfaced the defect: it caught exactly the two affected records — four stub tokens between them — and left every legitimate placeholder form in place. A subsequent production run confirmed the end-to-end behavior: the gate fired on the run’s two leaked records, overgeneration backfilled them, and the delivered dataset contained zero stub tokens. The rejection is recorded in the run manifest’s `rejection_counts` object as a first-class `template_leak` tally alongside `hallucination`, `privacy`, `bias`, `gdpr`, `url`, and `fidelity`. Adding a counted rejection reason changes the manifest’s shape, so the manifest schema advanced from 1.1 to **1.2** — adding `template_leak` to the required `rejection_counts` keys — and the orchestrator now emits `manifest_schema_version: "1.2"`. The version bump is the point: a buyer diffing two manifests sees a documented schema change explaining why the rejection shape differs, rather than a silently-added field. The machine schema (`run_manifest.schema.json`, JSON Schema 2020-12) is validated against freshly-generated manifests by a conformance test, so the documented shape and the emitted shape cannot drift.

The honest bound on this gate matches the one the original paper states for pattern-based compliance (Hill, 2026): the matching pattern is calibrated to the stub forms observed to date. A future run could surface a stub shape the pattern does not match — a form such as `<TODO_FILL_IN>` — which would require a one-line addition to the pattern. The gate is a reliable catch for a known, recurring artifact class, not a proof that no stub can ever pass.

5 Operational Hardening: Diagnosing and Resolving a Substrate Fault

The original paper (Hill, 2026) bounds the local substrate by its single GPU. This section documents a different local-substrate limitation that was not a throughput bound but a stability fault — one that, left undiagnosed, silently degraded both reliability and cost — and the controlled process by which it was isolated and permanently resolved. It is included because the diagnosis crossed the boundary between software and hardware, and because the resolution is the kind of full-stack operational competence that a methodology paper’s claims ultimately rest on: a pipeline is only as trustworthy as the substrate it runs on.

5.1 The Symptom

The local substrate began aborting intermittently with a C-level memory fault — `free(): invalid pointer`, followed by `Aborted (core dumped)`. The fault was non-deterministic. The same run, with the same inputs, would complete on one attempt and abort on another. Across a working session the application log recorded seventy-one run starts against twenty-six clean completions; the failure surfaced most often at the largest fresh memory allocation in the pipeline, which after the learned-NER tier was added became the GLiNER2 (Zaratiana et al., 2024, 2025) DeBERTa-large (He et al., 2020, 2021) model load in the de-identification evaluation step.

The temptation in this situation is to attribute the crash to the most recently added software — the learned-model load at whose moment the crash appeared. That attribution would have been wrong, and resisting it was the first step of the diagnosis.

5.2 Isolating Hardware from Software

The discriminating property of the fault was its non-determinism. A software defect in the model-load path would fail deterministically: the same code on the same inputs would crash every time. This fault did not. The model load that aborted on one attempt succeeded on the next with nothing changed. Tested across process boundaries, across resume and fresh runs, and across batch sizes, the crashes and the successes were mixed with no software discriminator — the signature not of a code defect but of marginal hardware surfacing under heavy allocation.

The diagnostic discipline was to make the fault measurable before attempting any fix. A controlled baseline — the crash-prone operation run repeatedly with its exit status captured — established the failure rate at approximately thirty percent. With a measured baseline, any candidate fix could be evaluated against it rather than declared successful after a single lucky run, which for a non-deterministic fault proves nothing.

Two software changes were made during this period, neither of which was a fix for the fault but both of which were correct hardening. First, the de-identification auto-evaluation was moved into a subprocess, matching the instruction path’s existing isolation. Because the crash occurred after generation had completed and the records were written, isolating the evaluation meant a memory fault in the learned-model load could no longer take down the parent process or the completed run; the evaluation became independently retryable, and no generated data could be lost to an evaluation-time abort. This containment rests on a durability property the pipeline already had: each batch is written to `runs/<run>/batches/` as accepted/rejected JSONL plus a `status.json` and `fsync’d` before the next batch begins, alongside the consolidated `output.jsonl`. A crash therefore cannot corrupt or lose completed batches, and `--resume` rebuilds the consolidated files from the durable per-batch artifacts — crash-safe and duplicate-free, requiring only the fixed seed that makes batch composition deterministic. The fault could abort a process, but the durable checkpoint and the frozen `resolved_config.yaml` plus `config_hash.txt` meant a resumed run continued exactly where it stopped. Second — unrelated to this fault but adjacent — the template-leak gate of Section 4 was added. The subprocess isolation did not reduce the crash rate; it contained the blast radius, which is a different and also necessary property. The root-cause of the fault remained to be determined.

5.3 Root Cause and Permanent Resolution

The host is an Intel Core i9-14900KF. That processor falls within the population affected by the documented Intel 13th- and 14th-generation desktop instability issue, which Intel attributes to a Vmin Shift localized to a clock-tree circuit within the IA core that is vulnerable to reliability

aging under elevated voltage and temperature, and whose symptom class is exactly this one: intermittent, allocation-triggered faults that worsen with age (Intel Corporation, 2024). The resolution was the vendor-supplied remediation — a motherboard firmware update to MSI BIOS A.90 (April 2026), which carries the Intel microcode revision (0x12B) that corrects the elevated-voltage behavior, applied together with Intel’s recommended default power settings (Micro-Star International, 2026).

The fix is permanent and verified. Following the firmware update, the pipeline completed nine consecutive full thousand-record runs across both substrates with zero faults, including the GLiNER2 DeBERTa-large loads on both Studio and Cloud evaluation paths — the exact operation that had been the most reliable trigger of the crash. Against the measured pre-fix failure rate of approximately thirty percent, nine consecutive clean completions is not a hopeful streak; it is a resolved fault. The `free(): invalid pointer` condition has not recurred.

5.4 The Fault Was Also a Cost

The fault’s most easily overlooked consequence was economic, and it connects this section directly to the next. On the cloud substrate, a mid-run abort forced a resume, and each resume paid another A10G container cold-start and model load before generation could continue. Every crash therefore inflated the realized cost of a run above its true compute cost by the price of an extra cold start. The firmware fix, by allowing runs to complete in a single pass, removed that resume overhead. The cost figures in Section 6 show the result: realized cost returning to the measured steady-state economics the original paper reported (Hill, 2026), no longer eroded by a stability tax. The hardening effort was not only a reliability fix; it restored the cost structure to its measured baseline.

6 Cloud Economics After Consolidation

The original paper reports the measured AWQ-vLLM economics of the cloud substrate on an A10G GPU (Hill, 2026). Two things have changed since that warrant a brief economic addendum: the three separate repositories were consolidated into a single application with a shared core, and the stability fault of Section 5 was resolved. The question this section answers is whether either change moved the per-record cost — and the answer is that consolidation did not raise it, and the fix restored it.

6.1 Consolidation Did Not Raise Per-Record Cost

The consolidation unified `syntheval_studio`, `syntheval_cloud`, and the shared logic into a single application around one `syntheval_core`. A reasonable concern is whether routing both substrates through a shared core introduced per-record overhead — whether the abstraction that made the system one application cost something at runtime. It did not, and the reason is structural: the substrate seam is a single backend protocol, not a translation layer. The shared core does not sit in the inference path adding work per record; it supplies the gating, provenance, evaluation, and delivery logic that is identical regardless of substrate, while each substrate’s backend performs generation directly. The cost of a record on the cloud substrate is the cost of A10G inference for that record, the same as it was before consolidation.

6.2 Measured Steady-State Cost

Per-run cost was measured directly from the Modal credit balance before and after each run; the cost of a run is the decrement. The representative figure is the most recent thousand-record run, which completed in a single pass with no resume:

A thousand-record cloud run costs approximately \$0.47 — about \$0.00047 per record, or \$0.47 per thousand.

This figure lands squarely inside the original paper’s measured AWQ-vLLM band of roughly \$0.43–\$0.48 per thousand records (Hill, 2026), which is the point: the original measurement was correct, and post-hardening realized cost matches it.

6.3 The Resume Tax, and Why the Last Run Is the Honest Figure

Earlier thousand-record runs in the same series cost more — between roughly \$0.55 and \$0.84 — and reporting an average across all of them would overstate the true cost. The higher figures are not a different compute cost; they are the stability fault of Section 5 expressed in dollars. Each mid-run abort forced a resume, and each resume paid another A10G cold-start and model load. The earlier runs carried that resume tax; after the firmware fix each run, completing in a single pass, does not. The honest representative figure is therefore the no-resume run, not the average — the steady-state cost the system incurs now that runs complete reliably.

A second, smaller amortization effect is worth stating for completeness. The hundred-record runs in the same series show a higher per-thousand cost — on the order of several dollars per thousand rather than fifty cents — because the fixed overhead of container spin-up and model load is amortized over only a tenth as many records. This is the expected behavior of a fixed-overhead-plus-marginal cost structure, and it is the concrete economic reason the system routes large deliverables to the cloud substrate and reserves small local runs for sample generation: cost per record falls as run size rises, and the thousand-record scale is where the marginal A10G economics dominate the fixed overhead.

The combined economic picture is that consolidation into one application preserved the measured per-record cloud cost, and resolving the substrate fault restored realized cost to that measured figure by eliminating the resume tax. The system is both one application and as cheap to run as it was as three.

7 Delivery, Manifest, and Provenance

The contributions above describe what the system produces and how it is scored. This section describes how a scored run becomes an auditable, sellable artifact — the packaging and provenance machinery that turns generated records into a versioned deliverable a regulated buyer can verify. For a product whose differentiator is transparency, this machinery is not incidental; it is the part of the system that makes the rest of the claims checkable.

7.1 The Run Manifest as the Provenance Record

Every run writes a single authoritative record, `run_manifest.json`, validated against a published JSON Schema (`run_manifest.schema.json`, JSON Schema 2020-12) by a conformance test so the documented shape and the emitted shape cannot drift. The manifest is self-describing: it records the run identity and mode, the product and labeling axes, the full resolved configuration inline, and — for auditability — a provenance object and a reconciliation object.

The provenance object captures both *code* and *environment*: the git commit, branch, and whether the working tree was dirty; and the Python, platform, and best-effort torch, transformers, vllm, modal, and cuda versions. A buyer can therefore tie a delivered dataset to the exact code state and runtime that produced it. The reconciliation object is a cross-check the evaluation pipeline verifies on load: the count of accepted record identifiers must equal `total_accepted`, and the domain distribution must sum consistently. Because every count

in the manifest derives from the accepted-records JSONL as the single source of truth, the manifest, the evaluation report, and `output.jsonl` cannot disagree.

The manifest schema is itself versioned, and its version history is part of the provenance story: 1.0 carried the original counts and reconciliation; 1.1 added the provenance, `resolved_config`, and checkpointing objects and the product / labeling product axis; and 1.2 — introduced with the work in this addendum — added the `template_leak` rejection reason described in Section 4. A versioned schema means a change in manifest shape is always a documented event, not a silent one.

7.2 The Delivery Packager

`DeliveryPackager` (in `syntheval_core/delivery/`) turns a completed run into a versioned SKU under `<run>/delivery/<sku>_<version>/`. The package is self-describing and buyer-facing, assembled from components that each map to a module in the delivery line: a deterministic 70/20/10 train/validation/test split stratified by domain (`splitter.py`); a free sample; a product-aware `data_card.pdf` and an executive `customer_summary.pdf` rendered from the manifest and the evaluation report (`cards.py`); the evaluation report itself; a SHA-256 `checksum_manifest.json` over every delivered file (`checksums.py`); a `CHANGELOG.md` and SKU version parsing (`versioning.py`); and license and README text.

Two properties of the packager matter for the regulated-buyer audit story. First, the stratified split is deterministic: the same run and seed produce the same split, so a buyer can reproduce the partition rather than trust it. Second, a dirty git working tree blocks a release unless explicitly overridden — clean provenance is treated as part of the product, so a deliverable cannot be cut from an unversioned code state by accident. The checksum manifest closes the loop: a buyer can verify that the files received are bit-for-bit the files the manifest describes.

7.3 Why This Is the Product

A construction-true de-identification dataset is only as valuable as a buyer’s ability to trust its labels, and trust in a synthetic-data product is built from exactly these artifacts: a manifest that ties the data to its code and configuration, an evaluation report that states per-type detector performance against the construction-true ground truth, a reproducible split, and a checksum manifest that proves integrity. The packaging machinery is therefore not separate from the product — it is the form in which the product’s central claim, that its labels are correct by construction and its quality is measured rather than asserted, is delivered to someone who must verify it before use.

8 Limitations

This addendum inherits every limitation the original paper (Hill, 2026) states, and adds a small number specific to the work documented here. They are stated plainly, in the original’s register: the value of a methodology is bounded by the honesty of its limitations.

8.1 The Premium Tier’s Precision Cost Is Real and Concentrated

The gap-only hybrid recovers free-text-entity recall at a measurable precision cost on exactly the types it recovers. In the representative run, the learned model produced 141 false positives across the two gap types — over-tagging spans as personal names or institution identifiers — to recover entities the regex detector cannot find at all. This is a favorable trade for a premium tier whose purpose is closing the free-text gap, and it is reported per type rather than hidden in an aggregate, so a buyer sees precisely what the learned model costs and

what it buys. But it is a cost, not a free improvement: the learned model’s recall on free-text entities comes with a higher false-positive rate on those same entities than the deterministic detector exhibits anywhere. A consumer who needs maximal precision over maximal recall on names should weight the base regex tier accordingly.

8.2 The Gap Set Is Empirical, Not Derived

The final gap set — personal names and institution identifiers — was found by measurement, through two narrowings that each removed types on which the learned model did not strictly outperform regex. This is a strength in that the scope is grounded in measured behavior rather than assumption, but it is also a limitation: the gap set is specific to the entity types and the value distributions observed in this product’s domains. A new domain with a different free-text entity type, or a different distribution of an existing one, could shift which types the learned model strictly improves, and the correct gap set for that domain would have to be re-established empirically rather than assumed from this one.

8.3 The Template-Leak Pattern Is Calibrated, Not Complete

As Section 4 states, the template-leak gate matches the stub forms observed to date. It is a reliable catch for the known artifact class and is deliberately narrow to avoid degrading legitimate placeholders, but it is calibrated to seen forms, not proven complete against all possible ones. A novel stub shape would pass until the pattern is extended. This is the same bounded-completeness property the original paper states for all pattern-based detection in the system (Hill, 2026), applied to one additional pattern.

8.4 The Hardware Resolution Is Verified for This Host

The firmware fix is verified permanent for the affected host, established by nine consecutive fault-free full runs against a measured thirty-percent pre-fix failure rate. That evidence resolves the fault on this machine. It does not constitute a general claim about the processor family or about every affected unit; it is the verified resolution of a specific fault on a specific host through the documented vendor remediation. The general lesson the section offers is methodological — how a non-deterministic substrate fault was isolated from software and root-caused — rather than a universal hardware claim.

9 Conclusion

The work documented in this addendum extends SynthEval from a quality-gated instruction-data pipeline into a system that also produces and rigorously evaluates a construction-true de-identification product, and it does so without disturbing the results the original paper (Hill, 2026) established.

The four contributions form a coherent line. Construction-true labeling gives the de-identification product a ground truth that no detector’s recall ceiling bounds, which is what makes it suitable for training and benchmarking models against entities a deterministic detector cannot find. The learned-NER premium tier is the instrument that measures that closure, and its central design decision — restricting the learned model to the two entity types it strictly improves, found through two empirical narrowings including one configuration that failed instructively — is the addendum’s clearest example of the original paper’s discipline: scope established by measurement, with the failure reported as fully as the result. The template-leak gate removes an emergent generation defect at the point where prevention is actually possible, the finalized record, after establishing why prevention at the prompt is not. And the operational-hardening effort diagnosed a non-deterministic substrate

fault by isolating hardware from software, contained its blast radius in code, and resolved it permanently through the documented vendor remediation — restoring not only reliability but, by eliminating the resume tax, the measured cost structure as well. Underneath all four runs the same delivery and provenance machinery — the versioned manifest at schema 1.2, the reconciliation cross-check, the stratified split, and the checksum-sealed delivery package — is what makes each of these claims auditable rather than merely asserted, turning a scored run into a deliverable a regulated buyer can verify.

The thread connecting them is the one the original paper began with (Hill, 2026): a claim is worth no more than the evidence and the bounds stated alongside it. The premium tier’s recall recovery is reported with its precision cost per type; the template-leak gate is reported with the pattern’s calibration limit; the hardware fix is reported as verified for its host against a measured baseline, not generalized beyond it; and the cost figure is the honest no-resume steady state, not a flattering average. Read alongside the original methodology paper, this addendum is intended to leave a reader with the same thing the first paper aimed for — not a marketing impression of capability, but a precise, bounded, and auditable account of what the system does and what it costs.

10 References

- He, P., Gao, J., & Chen, W. (2021). *DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing* (arXiv:2111.09543). arXiv. <https://arxiv.org/abs/2111.09543>
- He, P., Liu, X., Gao, J., & Chen, W. (2020). *DeBERTa: Decoding-enhanced BERT with disentangled attention* (arXiv:2006.03654). arXiv. <https://arxiv.org/abs/2006.03654>
- Hill, P. (2026). *SynthEval Studio and SynthEval Cloud: Quality-gated synthetic data generation for regulated industries*. PBH Applied Systems LLC. <https://pbhappliedsystems.com>
- Intel Corporation. (2024, September). *Intel Core 13th and 14th Gen desktop instability root cause update*. Intel Community. <https://community.intel.com/t5/Blogs/Tech-Innovation/Client/Intel-Core-13th-and-14th-Gen-Desktop-Instability-Root-Cause/post/1633239>
- Micro-Star International. (2026, April). *BIOS release A.90 (Intel microcode 0x12B)* [Motherboard firmware release notes]. <https://www.msi.com>
- Zaratiana, U., Tomeh, N., Holat, P., & Charnois, T. (2024). GLiNER: Generalist model for named entity recognition using bidirectional transformer. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (pp. 5364–5376). Association for Computational Linguistics. <https://arxiv.org/abs/2311.08526>
- Zaratiana, U., Pasternak, G., Boyd, O., Hurn-Maloney, G., & Lewis, A. (2025). *GLiNER2: An efficient multi-task information extraction system with schema-driven interface* (arXiv:2507.18546). arXiv. <https://arxiv.org/abs/2507.18546>